SCC.351: Advanced Cyber Security
Lab 3: Secret Sharing, Threshold Cryptography and Zero-Knowledge Proofs

Task 1: Adding an Election Administrator to Your Voting System

In this task, you will add an election administrator to your voting system so that votes are accepted only when the election is open and results are released only when it is closed.

Your modified system will consist of four components:

- **Admin**: open, close and query election state.
- **Registrar**: generates key pairs for eligible voters, distributes signing keys and stores public verification keys for server access.
- **Client**: signs votes using the voter's ECDSA private key, submits signed votes and requests status or results.
- **Server**: accepts and verifies signed votes only while the election is open, records the tally, and returns the result only when the election is closed.

Question 1
Modify your data flow diagram from lab 2 to incorporate the administrator.

Question 2
Write a script for the administrator using the skeleton file provided on Moodle (`admin.py`) that implements three actions: open, close and status.

Question 3
Modify your server.py and client.py files from lab 2 as follows.

Your client.py file should:

- Query the server status before each action.
- Refuse to submit a vote when the server state is closed and return an error.
- Refuse to request results when the server state is open and return an error.

Your server.py file should:

- Maintain election state.
- Accept votes only when the election is open.
- Store votes and tallies while open.
- Return the result only when the election is closed.

You will not need to make changes to your registrar.py file.

You should be able to perform the following actions in the order given:

1. Start the voting server.
2. Run the registrar to create keys for voters.

3. Administrator opens the election.
4. Voting client can vote and check the status of the election.
5. Administrator closes the election.
6. Voting client can retrieve results.

Voting clients should be unable to vote if the election is closed, and should be unable to retrieve results if the election is open.

Task 2: Threshold RSA

In this task, you will implement threshold RSA using Shamir Secret Sharing.

Question 1
Implement Shamir Secret Sharing using the file (`shamir.py`) provided on Moodle.

Your implementation must support the following:

- Creation of n shares of a secret s.
- Reconstruction of the secret when provided with at least t shares.
- Demonstrate failure cases: show that reconstruction does not recover the secret when fewer than t shares are supplied, and show that reconstruction fails or returns an incorrect value if one or more shares are tampered with.

Question 2
Write a script that implements threshold RSA encryption using the skeleton file provided on Moodle (`rsa-shamir.py`). Your script should use your `shamir.py` functions to split and reconstruct the RSA private decryption key.

Question 3
Explain how you would integrate your threshold RSA solution into the voting system from Task 1. For each component (Admin, Registrar, Client, Server), state the concrete changes required. Outline the end-to-end flow from key generation to encryption, vote submission, reconstruction of the decryption key on the server, and final decryption and tallying.

Task 3: Zero-Knowledge Proofs

Question 1
Explain briefly where zero-knowledge proofs could be usefully applied in your voting system from task 1. For each place you identify, you should give a short explanation of what the proof would assert and why the property is useful. In your answer, you should identify at least three places where zero-knowledge proofs could be usefully applied.

Question 2
Alice has two balls, one red and one green, which are wrapped identically so others cannot see their colours. Alice can distinguish the two balls. Bob wants assurance that Alice can distinguish the balls, but Alice does not want to reveal which ball is red and which is green.

Design an interactive protocol that Alice and Bob can use to achieve this.
(Hint: you may want to revisit the Cave of Secrets example from Lecture 6).

Describe the protocol and then give a short explanation of why this protocol satisfies the following properties:

- Completeness: why an honest Alice who can distinguish the balls can always convince an honest Bob.
- Soundness: why a cheating Alice who cannot distinguish the balls will be detected by Bob with high probability.
- Zero-knowledge: why the protocol does not reveal which ball is red and which is green.